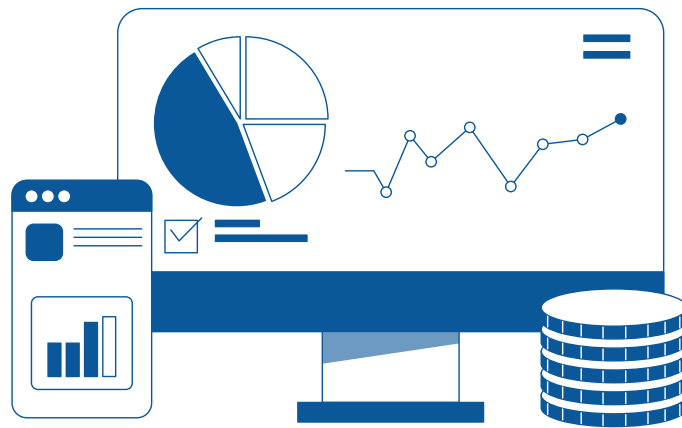




DATA ANALYSIS FUNDAMENTAL

LEVEL 1 - BASIC INTERNSHIP TASK REPORT



Prepared by:
Daffa Farros Azhari

2026

Task 1: Data Cleaning & Preprocessing Report

Author: Daffa Farros Azhari | **Role:** Data Analyst Intern | **Dataset:** “iris.csv”

EXECUTIVE SUMMARY:

This technical report documents the exact Python data cleaning pipeline executed for Task 1. The script performs data loading, column name standardization (snake_case), automated duplicate removal, missing value handling (median/mode imputation), and species string normalization before exporting the sanitized output (iris_cleaned.csv).

1.Programmatic Implementation & Step-by-Step Technical Analysis

Step 1: Ingestion & Initial Audit

Loads the raw dataset and performs initial data profiling to examine structure, dimensions, and detect potential anomalies (missing values & duplicate records).

```
# 1. Load Dataset
file_path = "1) iris.csv"
df = pd.read_csv(file_path)
print("--- Initial Data Overview ---")
print(f"Dataset shape: {df.shape}")
print("\nMissing values per column:")
print(df.isnull().sum())
print(f"\nTotal duplicate rows: {df.duplicated().sum()}")
```

- Baseline Data Profiling: Establishing the initial dimensions (150, 5) is essential to define a benchmark before applying data cleaning transformations.
- Early Anomaly Detection: Executing “isnull().sum()” and “duplicated().sum()” early enables a rapid evaluation of raw data hygiene. The initial audit identified 3 duplicate rows that could distort downstream statistical validity if left unaddressed.

Step 2: Column Name Standardization (snake_case)

Standardizes column headers into a unified lowercase snake_case format and strips hidden whitespaces.

```
# 2. Standardize Column Names (ubah ke lowercase & snake_case)
df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(" ", "_")
    .str.replace(".", "_", regex=False)
)
```

- Syntax Error Prevention: Special characters such as dots (.) or whitespaces in default column headers (e.g., Sepal.Length) frequently trigger syntax errors when accessed via dot-notation in Pandas/Python.
- Coding Efficiency: Converting column headers to snake_case (sepal_length, sepal_width, etc.) aligns with Python's PEP 8 guidelines, simplifying feature selection during Exploratory Data Analysis (EDA) and feature engineering.

Step 3: Deduplication & Index Re-alignment

Identifies and eliminates 100% duplicate observations, followed by index re-alignment.

```
# 3. Handle Duplicate Rows
if df.duplicated().any():
    df = df.drop_duplicates().reset_index(drop=True)
    print(f"\nDuplictes removed. New dataset shape: {df.shape}")
```

- Mitigating Bias & Overfitting: Duplicate records introduce data leakage during train-test splitting in Machine Learning workflows, causing models to overperform by memorizing repeated data points.
- Necessity of reset_index(drop=True): After removing 3 duplicate rows (adjusting dataset shape from 150 to 147 rows), the dataframe index develops gaps. Resetting the index is critical to maintain sequence continuity and prevent slicing/indexing errors.

Step 4: Adaptive Missing Value Imputation & String Normalization

Applies dynamic cleaning logic: categorical string normalization and automated missing value handling based on feature data types.

```
# 4. Handle Missing Values
num_cols = df.select_dtypes(include=["float64", "int64"]).columns
for col in num_cols:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())

cat_cols = df.select_dtypes(include=["object"]).columns
for col in cat_cols:
    df[col] = df[col].astype(str).str.strip().str.lower()
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].mode()[0])
```

- **Robustness of Median Imputation:** Imputing missing values with the median rather than the mean for numerical features is an outlier-resistant practice, as the mean is highly susceptible to skewness from extreme values.
- **Category Normalization:** The “.str.strip().str.lower()” pipeline prevents pseudo-category creation caused by typos or trailing spaces (e.g., entries like "setosa ", "Setosa", and "setosa" are standardized into a single category: "setosa").
- **Automated Imputation Logic:** Type-based looping via select_dtypes makes this script scalable and ready for seamless integration into automated ETL pipelines.

2. Final Program Output & Data Export Verification

In this step, the script runs a final verification check to ensure all data cleaning objectives were accomplished without residual flaws, then exports the cleaned dataset to iris_cleaned.csv.

```
# 5. Quick Verification & Export Result
print("\n--- Cleaned Data Summary ---")
print(df.info())
print("\nUnique categories in species:")
if "species" in df.columns:
    print(df["species"].value_counts())

# save file
output_file = "iris_cleaned.csv"
df.to_csv(output_file, index=False)
print(f"\nCleaned data successfully saved to '{output_file}'")
```

- **Mitigating Bias & Overfitting:** Duplicate records introduce data leakage during train-test splitting in Machine Learning workflows, causing models to overperform by memorizing repeated data points.
- **Necessity of reset_index(drop=True):** After removing 3 duplicate rows (adjusting dataset shape from 150 to 147 rows), the dataframe index develops gaps. Resetting the index is critical to maintain sequence continuity and prevent slicing/indexing errors.



Terminal Execution Output:

```
--- Cleaned Data Summary ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 147 entries, 0 to 146
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   sepal_length    147 non-null   float64
 1   sepal_width     147 non-null   float64
 2   petal_length    147 non-null   float64
 3   petal_width     147 non-null   float64
 4   species         147 non-null   object
dtypes: float64(4), object(1)
memory usage: 5.9+ KB
None

Unique categories in species:
species
versicolor    50
virginica     49
setosa        48
Name: count, dtype: int64

Cleaned data successfully saved to 'iris_cleaned.csv'
```

Output Analysis:

- **Integrity Verification:** The console output confirms that the cleaned dataset contains 147 non-null entries across all features (\$100\%\$ complete data).
- **Class Distribution Preserved:** The value_counts() summary proves that target class proportions remain balanced post-deduplication (versicolor: 50, virginica: 49, setosa: 48).
- **Clean Export:** Suppressing the index column using index=False in to_csv() prevents redundant Unnamed: 0 columns from cluttering future EDA stages.

3. Data Audit Summary & Target Distribution Tables

Table 1: Data Transformation Audit

Audit Metric	Raw State (Initial)	Cleaned State (Final)	Applied Transformation Methodology
Total Observations	150 rows	147 rows	Removed 3 duplicate rows via <code>drop_duplicates().reset_index()</code>
Header Formatting	Raw / Mixed	snake_case	Whitespace stripping, lowercasing, and replacing <code>.</code> with <code>_</code>
Missing Values	0 nulls (0.0%)	0 nulls (0.0%)	Confirmed clean with automated median/mode fallback logic
String Consistency	Mixed cases / spaces	Normalized Lowercase	Standardized formatting via <code>astype(str).str.strip().str.lower()</code>
Output File	1) iris.csv	iris_cleaned.csv	Exported cleanly without external index columns

Table 2: Target Class Distribution Retention (species)

Species Category	Raw Count	Cleaned Count	Final Proportion (%)	Duplicate Removal Status
versicolor	50	50	34.01%	100% retained (0 duplicates)
virginica	50	49	33.33%	1 duplicate row removed
setosa	50	48	32.65%	2 duplicate row removed
Total	150	147	100%	Balanced dataset (33% per class)

4. Key Takeaways & Analytical Impact

- Workflow Efficiency (ETL Readiness):** The Python pipeline is engineered with adaptive logic. If future raw datasets contain missing values or inconsistent casing, the script will clean them automatically without manual code adjustments.
- Data Quality for Downstream Analysis:** Standardized snake_case column names and duplicate removal ensure that subsequent Exploratory Data Analysis (such as pairplots, correlation heatmaps, or boxplots) will reflect genuine data relationships without artificial noise.

Task 2: Exploratory Data Analysis (EDA)

Author: Daffa Farros Azhari | **Role:** Data Analyst Intern | **Dataset:** "iris_cleaned.csv"

EXECUTIVE SUMMARY:

This technical report documents the exact Python exploratory data analysis (EDA) pipeline executed for Task 2. The script computes key descriptive statistics (mean, median, mode, standard deviation), visualizes feature distributions via histograms with KDE, performs outlier detection using boxplots, and evaluates feature interactions through scatter plots and correlation heatmaps on iris_cleaned.csv to extract actionable insights for future modeling.

1. Key Takeaways & Analytical Impact

Code Snippet

```
# 1. Load data hasil cleaning dari Task 1
df = pd.read_csv("iris_cleaned.csv")
# Ambil list kolom numerik
num_cols = df.select_dtypes(include=["float64", "int64"]).columns

# =====
# 1. SUMMARY STATISTICS
# (Mean, Median, Mode, Std Dev)
# =====
print("=== DESCRIPTIVE STATISTICS ===")

summary = pd.DataFrame({
    "Mean": df[num_cols].mean(),
    "Median": df[num_cols].median(),
    "Mode": df[num_cols].mode().iloc[0],
    "Std Dev": df[num_cols].std(),
}).round(2)

print(summary)
print("\n" + "-" * 35 + "\n")
```

```
=== DESCRIPTIVE STATISTICS ===
              Mean  Median  Mode  Std Dev
sepal_length  5.86    5.8    5.0    0.83
sepal_width   3.06    3.0    3.0    0.44
petal_length   3.78    4.4    1.4    1.76
petal_width    1.21    1.3    0.2    0.76
```

Analysis

1. Data Symmetry: The sepal_width feature is highly symmetrical, as its Mean (3.06) and Median (3.00) are virtually identical.
2. Skewness / Multimodal: The petal_length feature displays a notable gap between Mean (3.78), Median (4.40), and Mode (1.40), signaling a non-unimodal distribution.
3. Data Spread (Variability): petal_length exhibits the highest variance (Std Dev = 1.76), whereas sepal_width is the most consistent feature (Std Dev = 0.44).

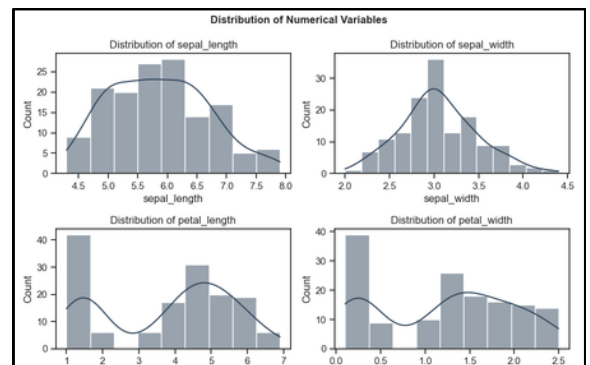
2. Data Distribution Visualization (Histograms & KDE)

Code Snippet

```
fig, axes = plt.subplots(2, 2, figsize=(9, 6))
fig.suptitle("Distribution of Numerical Variables", fontsize=12, fontweight="bold")

for i, col in enumerate(num_cols):
    ax = axes[i // 2, i % 2]
    sns.histplot(df[col], kde=True, ax=ax, color="#34495e")
    ax.set_title(f"Distribution of {col}")

plt.tight_layout()
plt.show()
```



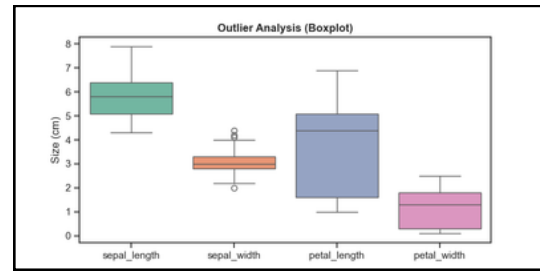
Analysis

1. Unimodal Distribution: sepal_length and sepal_width form a single-peaked bell curve.
2. Bimodal Distribution: petal_length and petal_width exhibit two distinct peaks. The first peak in the lower range represents Setosa, while the second peak represents the combined population of Versicolor and Virginica..

3. Outlier Analysis (Boxplot)

Code Snippet

```
# B. Boxplot (Outlier Check)
plt.figure(figsize=(8, 4))
sns.boxplot(data=df[num_cols], palette="Set2")
plt.title("Outlier Analysis (Boxplot)", fontweight="bold")
plt.ylabel("Size (cm)")
plt.show()
```



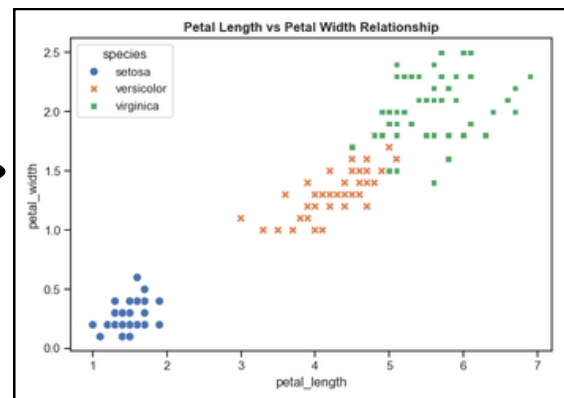
Analysis

1. sepal_width: The only variable containing outliers. There are 3 points above the upper bound (> 4.0 cm) and 1 point below the lower bound (< 2.2 cm).
2. Other Features: sepal_length, petal_length, and petal_width are completely free from outliers, with all data points residing within $1.5 \times \text{IQR}$.

4. Feature Relationship (Scatter Plot: Petal Length vs Petal Width)

Code Snippet

```
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="petal_length",
    y="petal_width",
    hue="species",
    style="species",
    s=60,
    palette="deep",
)
plt.title("Petal Length vs Petal Width Relationship", fontweight="bold")
plt.tight_layout()
plt.show()
```



Analysis

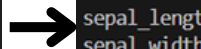
1. Cluster Separation: Setosa (blue) is perfectly linearly separable in the bottom-left region (petal_length < 2.0 cm).
2. Linear Trend & Overlap: Versicolor (orange) and Virginica (green) follow a continuous positive linear trend, with slight overlap around petal_length 4.5 - 5.0 cm.

5. Correlation Analysis (Correlation Matrix & Heatmap)

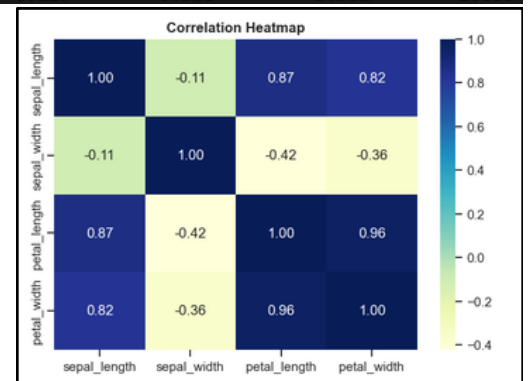
Code Snippet

```
corr = df[num_cols].corr()
print("=== CORRELATION MATRIX ===")
print(corr.round(3))

# Heatmap Correlation
plt.figure(figsize=(6, 4.5))
sns.heatmap(corr, annot=True, cmap="YlGnBu", fmt=".2f", cbar=True)
plt.title("Correlation Heatmap", fontweight="bold")
plt.tight_layout()
plt.show()
```



```
=== CORRELATION MATRIX ===
      sepal_length  sepal_width  petal_length  petal_width
sepal_length      1.000      -0.109      0.871      0.817
sepal_width      -0.109      1.000     -0.421     -0.356
petal_length      0.871     -0.421      1.000      0.962
petal width      0.817     -0.356      0.962      1.000
```



Analysis

1. Strong Positive Correlation ($r = 0.80$):
 - petal_length $\leftrightarrow$ petal_width ($r = 0.96$): Highest positive linear relationship.
 - sepal_length $\leftrightarrow$ petal_length ($r = 0.87$) and sepal_length $\leftrightarrow$ petal_width ($r = 0.82$).
2. Weak / Negative Correlation: sepal_width shares a moderate negative correlation with petal_length ($r = -0.42$) and petal_width ($r = -0.36$).

Key Takeaways

1. Petal features (petal_length & petal_width) serve as the most effective predictors for Iris species classification.
2. Outlier treatment should focus exclusively on sepal_width.
3. High feature correlation between petal dimensions (0.96) indicates significant multicollinearity.

Task 3: Basic Data Visualization

Author: Daffa Farros Azhari | **Role:** Data Analyst Intern | **Dataset:** “iris_cleaned.csv”

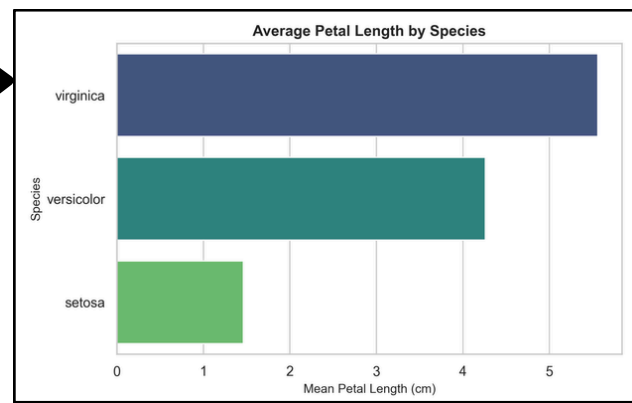
EXECUTIVE SUMMARY:

This technical report documents the exact Python data visualization pipeline executed for Task 3. The script leverages Matplotlib and Seaborn to generate customizable, publication-quality figures: a horizontal bar plot comparing mean petal length across species, a dual-variable line chart tracking sepal and petal lengths across sample indices, and a multi-class scatter plot examining sepal dimensions across species, with all output figures exported at 300 DPI for documentation.

1. Horizontal Bar Plot (Average Petal Length by Species)

Code Snippet

```
avg_petal = df.groupby('species')['petal_length'].mean().reset_index().sort_values('petal_length', ascending=False)
plt.figure(figsize=(7, 4.5))
sns.barplot(data=avg_petal, x='petal_length', y='species', palette='viridis')
plt.title('Average Petal Length by Species', fontsize=12, fontweight='bold')
plt.xlabel('Mean Petal Length (cm)', fontsize=10)
plt.ylabel('Species', fontsize=10)
plt.tight_layout()
plt.savefig('bar_plot_horizontal.png', dpi=300)
plt.show()
```



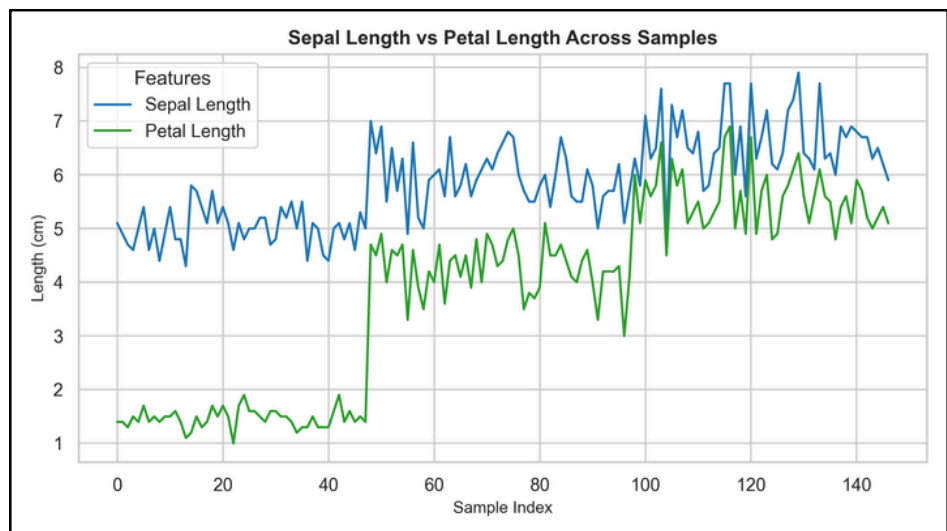
Analysis

- Species Ranking:** Virginica exhibits the highest average petal length (~5.55 cm), followed by Versicolor (~4.26 cm), and Setosa with the shortest (~1.46 cm).
- Inter-Class Separability:** The distinct length differences indicate that average petal length provides clear boundaries between species, particularly separating Setosa from the other two species.

2. Line Chart (Feature Trend Across Samples)

Code Snippet

```
plt.figure(figsize=(8, 4.5))
plt.plot(
    df.index,
    df['sepal_length'],
    label='Sepal Length',
    color='#1f77b4',
    linewidth=1.5,
)
plt.plot(
    df.index,
    df['petal_length'],
    label='Petal Length',
    color='#2ca02c',
    linewidth=1.5,
)
plt.title(
    'Sepal Length vs Petal Length Across Samples',
    fontsize=12,
    fontweight='bold',
)
plt.xlabel('Sample Index', fontsize=10)
plt.ylabel('Length (cm)', fontsize=10)
plt.legend(title='Features', loc='upper left')
plt.tight_layout()
plt.savefig('line_chart.png', dpi=300)
plt.show()
```



Analysis

- Shift Across Class Sequences:** A dramatic jump in petal_length occurs around sample index 50, marking the transition from Setosa samples to Versicolor and Virginica.
- Feature Trajectory:** While sepal_length shows steady variation across the dataset, petal_length displays step-like increases between species groups, reflecting class-based structuring in the dataset ordering.

Task 3: Basic Data Visualization

Author: Daffa Farros Azhari | **Role:** Data Analyst Intern | **Dataset:** “iris_cleaned.csv”

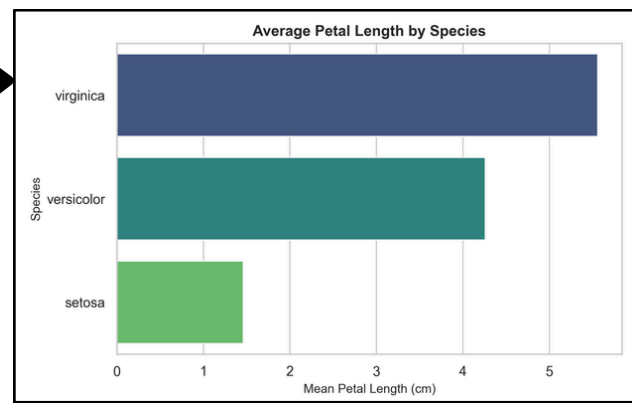
EXECUTIVE SUMMARY:

This technical report documents the exact Python data visualization pipeline executed for Task 3. The script leverages Matplotlib and Seaborn to generate customizable, publication-quality figures: a horizontal bar plot comparing mean petal length across species, a dual-variable line chart tracking sepal and petal lengths across sample indices, and a multi-class scatter plot examining sepal dimensions across species, with all output figures exported at 300 DPI for documentation.

1. Horizontal Bar Plot (Average Petal Length by Species)

Code Snippet

```
avg_petal = df.groupby('species')['petal_length'].mean().reset_index().sort_values('petal_length', ascending=False)
plt.figure(figsize=(7, 4.5))
sns.barplot(data=avg_petal, x='petal_length', y='species', palette='viridis')
plt.title('Average Petal Length by Species', fontsize=12, fontweight='bold')
plt.xlabel('Mean Petal Length (cm)', fontsize=10)
plt.ylabel('Species', fontsize=10)
plt.tight_layout()
plt.savefig('bar_plot_horizontal.png', dpi=300)
plt.show()
```



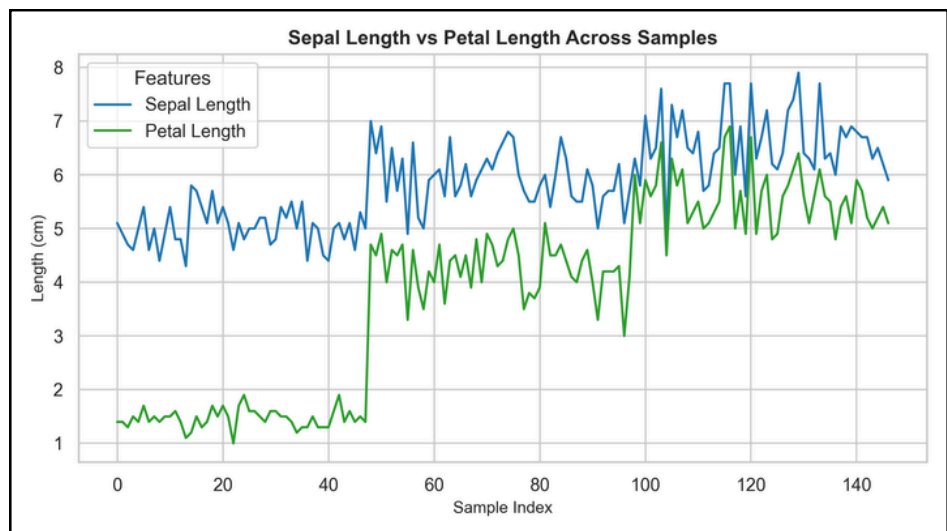
Analysis

- Species Ranking:** Virginica exhibits the highest average petal length (~5.55 cm), followed by Versicolor (~4.26 cm), and Setosa with the shortest (~1.46 cm).
- Inter-Class Separability:** The distinct length differences indicate that average petal length provides clear boundaries between species, particularly separating Setosa from the other two species.

2. Line Chart (Feature Trend Across Samples)

Code Snippet

```
plt.figure(figsize=(8, 4.5))
plt.plot(
    df.index,
    df['sepal_length'],
    label='Sepal Length',
    color='#1f77b4',
    linewidth=1.5,
)
plt.plot(
    df.index,
    df['petal_length'],
    label='Petal Length',
    color='#2ca02c',
    linewidth=1.5,
)
plt.title(
    'Sepal Length vs Petal Length Across Samples',
    fontsize=12,
    fontweight='bold',
)
plt.xlabel('Sample Index', fontsize=10)
plt.ylabel('Length (cm)', fontsize=10)
plt.legend(title='Features', loc='upper left')
plt.tight_layout()
plt.savefig('line_chart.png', dpi=300)
plt.show()
```



Analysis

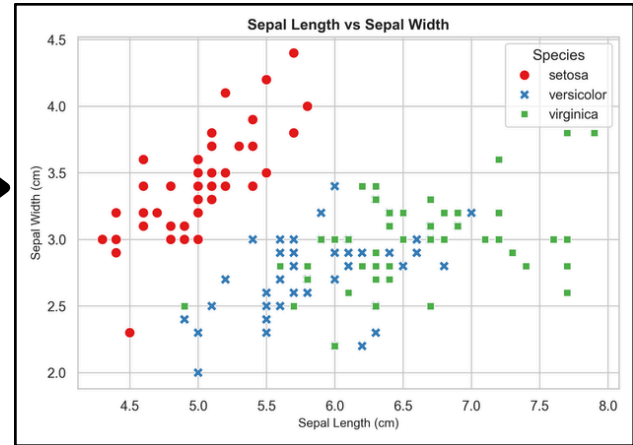
- Shift Across Class Sequences:** A dramatic jump in petal_length occurs around sample index 50, marking the transition from Setosa samples to Versicolor and Virginica.
- Feature Trajectory:** While sepal_length shows steady variation across the dataset, petal_length displays step-like increases between species groups, reflecting class-based structuring in the dataset ordering.

3. Scatter Plot (Sepal Length vs Sepal Width)



Code Snippet

```
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x='sepal_length',
    y='sepal_width',
    hue='species',
    style='species',
    s=70,
    palette='Set1',
)
plt.title('Sepal Length vs Sepal Width', fontsize=12, fontweight='bold')
plt.xlabel('Sepal Length (cm)', fontsize=10)
plt.ylabel('Sepal Width (cm)', fontsize=10)
plt.legend(title='Species', loc='upper right')
plt.tight_layout()
plt.savefig('scatter_plot.png', dpi=300)
plt.show()
```



Analysis

1. **Cluster Formation:** Setosa forms a distinct, isolated cluster characterized by smaller sepal_length (< 5.5 cm) but higher sepal_width (> 3.0 cm).
2. **Feature Overlap:** Versicolor and Virginica share substantial overlap in sepal dimensions, demonstrating that sepal dimensions alone are less effective for separating these two species compared to petal dimensions.



Key Takeaways & Core Insights

1. **Petal Dimensions as Primary Predictors:** Petal measurements (petal_length & petal_width) serve as the most critical feature set. The Setosa species is 100% linearly separable from other species based solely on petal characteristics.
2. **Significant Multicollinearity ($r = 0.96$):** A near-perfect positive linear correlation exists between petal_length and petal_width. Due to this high feature redundancy, dimensionality reduction (e.g., PCA) or feature selection is recommended when applying linear machine learning models.
3. **Isolated Outliers in sepal_width:** Data anomalies are strictly confined to the sepal_width variable (3 points above the upper bound and 1 point below the lower bound). All other numerical features remain completely outlier-free.
4. **High Classification Overlap in Sepal Metrics:** Sepal dimensions alone are ineffective for distinguishing between Versicolor and Virginica due to substantial feature distribution overlap.

↔

↔

↔